scientific reports

Check for updates

OPEN

Fluctuations of end-to-end delays in low-latency WebRTC-based UAV-borne internet of things operating in an urban environment

Agnieszka Chodorek[1], Robert R. Chodorek[2]✉ & Paweł Sitek[1]

Smart cities, first proposed in the early 1990s, continue to face new opportunities, such as the prevalence of UAVs, and new challenges, such as the need to provide hard real-time communications with UAVs with delays of sub-10-milliseconds and little jitter or even no jitter. Although existing highly reliable, low-latency wireless networks are capable of meeting these challenges, there is a general agreement that classical transport protocols cannot effectively meet such stringent requirements. This paper investigates UAV-borne WebRTC-based IoT for its ability to provide low jitter in an outdoor, highly reliable network environment. The results of field experiments conducted under various network conditions show that WebRTC was able to ensure little fluctuation of end-to-end delays. Moreover, in the case of error-free transmissions, fluctuations were so small that jitter, expressed in milliseconds, was close to zero. The significant advantage of the WebRTC Data Channel over the classic web logical channel, i.e. WebSocket, in terms of jitter was also demonstrated.

Keywords IEEE 802.11ac, Internet of things, Low latency, Smart cities, Unmanned aerial vehicle, WebRTC, WebSocket

With the development of smart cities, the demand for information obtained from the environment using an unmanned aerial vehicle (UAV)-enabled Internet of Things (IoT) is constantly growing. These ubiquitous devices enable the movement of loads to various corners of the city, operating both in a planned manner and ad hoc, anytime, anywhere. The use of UAVs within the city may be a convenient way to deliver parcels, and in the future also people, not only to a specific place, but also within a specific time window[1–3]. The high mobility of UAVs and their ability to operate in three dimensions have also made them convenient carriers of cameras and sensors for monitoring purposes[4,5]. Thanks to this, one of the first smart city services, monitoring and urban acquisition[6,7], has reached a higher level figuratively and literally.

A characteristic feature of smart city services is the acquisition of data in real time, which was already mentioned in the definition formulated a quarter of a century ago[6]. Providing real-time data is also used to improve the efficiency of services and coordinate urban activities[8]. For modern applications such as intelligent transportation system (ITS)[9,10], autonomous navigation infrastructure[11], remote control of mobile robots using video[12], digital twins[13,14], mobile edge computing (MEC)[15,16], etc., which inherently require reliable transmission, hard real time data is required. This often means end-to-end delays measured at the application level should be below 10 ms[10–12]. This contradicts the classic division of telecommunications traffic into elastic and inelastic, where the former must meet stringent time requirements and the latter must meet high reliability[17]. The traffic now has to balance stringent time requirements with high reliability.

If the IoT on board a UAV is performing monitoring, aerial MEC (AMEC), or other tasks for smart city services that require single-digit millisecond delays, both the air-to-ground link and the IoT protocol stack must provide such stringent delays. But it's not just about statistically small delays. Almost all or even all delays must be low. This means low jitter or even zero jitter. Although research on low-latency networks has already reached maturity, it is believed that the mechanisms of classical transport protocols are not able to effectively meet the requirements of such low delays[10]. The authors believe that Web Real-Time Communications (WebRTC) may be the solution to these problems.

[1]Department of Applied Computer Science, Faculty of Electrical Engineering, Automatic Control and Computer Science, Kielce University of Technology, Al. 1000-lecia P.P. 7, 25-314 Kielce, Poland. [2]Institute of Telecommunications and Cybersecurity, Faculty of Computer Science, Electronics and Telecommunications, AGH University of Krakow, Al. Mickiewicza 30, 30-059 Krakow, Poland. ✉email: chodorek@agh.edu.pl

While our previous work[18] focused on checking whether and to what extent IoT carried by a UAV, implementing data transmission based on the current WebRTC standard, was able to ensure low latencies in variable network conditions, this one is devoted to the jitter observed during transmissions carried out in such circumstances. The main contributions of this article are as follows:

- Delay measurements at the transport protocol level and at the web logical channel level during air-to-ground IoT transmission under variable network conditions and various environmental conditions, carried out using a universal framework[5].
- Analysis of measures of spread calculated from a series of end-to-end delays that have been measured during UAV- and WebRTC-based IoT transmissions in urban areas, under various network conditions and various weather conditions.
- Comparative analysis of measures of spread calculated from a series of end-to-end delay measurements carried out using a WebRTC-based IoT with the results obtained using a WebSocket-based reference system and solutions known from the literature.

The rest of this paper is as follows. First, a brief description of the state of the art in low-latency communication in smart cities is given. Then, low-latency UAV-based and WebRTC-based IoT intended for smart cities is discussed. Furthermore, field experiments aimed at measuring end-to-end delays in a highly variable network environment under various weather conditions are described. In the next section, the measures of spread calculated from the series generated during the experiments are presented and discussed. Then, the results discussed in this section are compared with those obtained for the reference system and those known from the literature. Finally, this paper is concluded and future work is proposed.

Low latency and low jitter in technologies of smart cities

For many years, smart cities have remained a hot research topic, mainly because the transition from a classic city to a smart city is treated as a way to improve the quality of life in the cities[6], including a way to mitigate problems resulting from the growth of the urban population and rapid urbanization[19]. The concept of a smart city was introduced in the early 1990s[20] and has continued to evolve[6,19,21]. As Yigitcanlar et al.[22] pointed out, although the initial rationale for the development of smart cities was mainly environmental issues, in current practice they receive only marginal attention and technology is at the core. The importance of digital engineering in the development of smart cities can be seen even in the definitions of smart cities. Definitions from a quarter of a century ago focused on urban information and monitoring[6,7]. Definitions from the past decade emphasize the role of information and communication technologies (ICT)[21,23]. The definition currently used by the European Commission[24] treats a smart city as a place where traditional services are improved using digital solutions, so the focus is on the digital transformation of traditional smart city services.

The newer definitions do not change but rather expand the older ones. The key services of modern smart cities still are traditional services using the IoT and ICT[23]: ITS, environmental and energy monitoring, health care systems, public safety systems and so on. These are often the same services that are identified as requiring highly reliable (Table 1), low-latency communications, and some require end-to-end delays of less than 10 milliseconds measured at the application level. Traditional services are supported by new, low-latency ones, such as digital twins[13,14,25], sometimes working with augmented reality (AR) or virtual reality (VR) systems. Another low-latency service is MEC[15,16,26], which moves processing closer to the data source. MEC supports traditional services by reducing IoT latencies[26]. Both traditional and new services require monitoring using sensors, often carried out using IoT technology. Sensors may be carried on board the UAV or the UAV may be the destination for transmitting sensor data. Examples of sensors on board UAVs are pollution monitoring[27,28],

Service	Reliability Constraint	Time Constraint	Paper
Health Monitoring	99.99%	100 ms	Rico and Merino[10]
Process automation (process monitoring)	99.99%	>50 ms	Mourtzis et al.[12]
Internet of Drones (remote control)	99.9%	from 5 to 50 ms	Rico and Merino[10]
Approaching autonomous navigation infrastructure	99.9%	10 ms	Purucker et al.[11]
Intelligent Transportation System	99.999%	from 5 to 10 ms	Rico and Merino[10]
AR/VR	99.99%	from 5 to 10 ms	Rico and Merino[10]
Smart Grids	99.999%	8 ms	Rico and Merino[10]
Transport Industry	99.999%	3 or 7 ms	Han et al.[34]
Power Distribution	99.9999%	3 or 6 ms	Han et al.[34]
AR/VR	99.999%	from 1 to 4 ms	Han et al.[34]
Cyber-Physical Systems	99.999%	from 0.5 to 2 ms	Rico and Merino[10]
Connecting autonomous vehicles	99.99999%	<1 ms	Giordani et al.[35]
eHealth	99.99999%	<1 ms	Giordani et al.[35]
Unmanned mobility	99.99999%	<1 ms	Giordani et al.[35]

Table 1. Selected low-latency services.

weather monitoring[29,30], monitoring of transport infrastructure[31,32]. A UAV with a computer on board was the destination of the Internet of Medical Things (IoMT) data while performing the AMEC service[33].

Selecting appropriate wireless communication technologies for UAV operations in a smart city is essential[36]. To ensure low latency communication, a low latency underlying network is necessary. Currently, sub-10 millisecond end-to-end delays are achievable in wireless networks built using the fifth-generation (5G) technology standard for cellular networks[37], Wi-Fi 7[38], for lower network load also Wi-Fi 6[17], and even Wi-Fi 5, which is capable of providing sub-10 millisecond handover latency due to Fast Roaming service[39]. If a smart city service requires deterministic traffic, i.e. traffic characterized by both low latency and no jitter, time-sensitive networking (TSN) technology should be used. TSN, originally intended for wired networks, has now been extended to 5G and Wi-Fi (Wi-Fi 6 and Wi-Fi 7 only)[40]. Networks that do not use TNS can only carry non-deterministic traffic, i.e. traffic requiring low latency and tolerant to low jitter.

The delays introduced by the underlying network cannot be reduced by higher network layers for obvious reasons. Protocol processing can only increase them. In the case of the network layer, Medeiros et al.[41] shown that classic wireless routing protocols operating in a mobile environment are unable to provide low end-to-end delays, low jitter and high reliability. To deal with this problem in smart cities, various solutions are proposed, including those based on Software-Defined Networking (SDN)[42,43], also used in IoT networks[44]. However, the basic solution is to move the processing close to IoT devices, for example using MEC, and then it is enough that the low-latency requirements are met only in the immediate vicinity of IoT devices.

In the transport layer, the classic Transmission Control Protocol (TCP) is considered unable to provide low latency[10]. This also applies to the TCP protocol that is part of the classic web logical channel, which is WebSocket. The other classic protocol, User Datagram Protocol (UDP), guarantees lightweight communication[44]. This simple, connectionless protocol is used as the underlying protocol for other transport protocols. The build-on-a-top of UDP protocols are Quick UDP Internet Connections (QUIC) and Stream Control Transmission Protocol (SCTP). While the former is considered a low-latency protocol[44–46], opinions on the latter are divided. There is an opinion that SCTP does not meet low latency requirements[10] and research showing that SCTP provides low latency when the underlying network allows it[18,47]. These research concerns the new version of the standard, i.e.[48], developed for WebRTC.

WebRTC[49] is a technology that allows web applications to communicate natively in real-time. The strength of WebRTC is that it provides a comprehensive communication service covering protocols from the transport layer to the presentation layer, thus ensuring session management, security, and transport of media and non-media data. Session management is similar to a proven architecture originally developed for desktop-class applications. Both media and non-media data transmissions are cryptographically protected using the Datagram Transport Layer Security (DTLS) protocol. The transport of media and non-media data takes place via two different web logical channels. The first one is intended for audio/video (inelastic) traffic, the second one for elastic traffic. The first one uses Real-time Transport Protocol (RTP) built on top of UDP, the second one uses SCTP. This second channel, named the Data Channel, is the functional equivalent of the WebSocket logical channel and can be used for IoT communications. However, while the typical WebSocket-based IoT architecture is Web of Things (WoT)[50], where transmission takes place via a WoT server, new WebRTC use cases include classic IoT[51].

The essence of low latency services is to achieve low end-to-end delay at the application level, so protocol processing time at the application layer is also important. The classic Message Queuing Telemetry Transport (MQ Telemetry Transport or MQTT) protocol, popular in the industry due to its low overhead and simpler communication model[52], proved to be suitable for low latency both when transmission was performed using QUIC[46,53] and WebRTC Data Channel[18]. The Generic Object Oriented Substation Events (GOOSE) protocol, used in smart grids, was also able to provide low latency[54,55]. GOOSE naturally introduces low jitter by delivering data in a 4 ms time window. In the application layer, custom protocols are often used in addition to general-purpose protocols. Jiang et al.[56] proposed the Low-latency Reliable Transmission (LRT) protocol, operating over UDP, to reduce delay and jitter when communicating with UAVs over a cellular network.

System

This section discusses the concept of a low-latency IoT system for monitoring in smart cities built using the UAV- and WebRTC-based universal framework[5], and shows the WebRTC-based IoT software and a reference system.

The UAV- and WebRTC-based universal framework as a low-latency IoT

The WebRTC-based universal framework to monitor urban areas, proposed by Chodorek et al.[5], consists of an air station and a ground station. The former consists of a UAV with good airworthiness, acting as a carrier of an efficient IoT system built on a Raspberry Pi 4 Model B single-board computer (SBC). The latter consists of two physically separate parts, which are the Command and Control Console (CCC), used to pilot the UAV, and the WebRTC Multimedia and Monitoring Station (WMMS). The WMMS handles ongoing tasks, such as online visualization of monitored factors, and stores data in a local storage. Typically, the WMMS connects to the air station via a Wi-Fi 5 connection using the network interface built into the SBC. Wi-Fi 5 is one of the wireless networking standards mentioned in the previous section that can provide low-latency communication services for non-deterministic traffic.

Two framework input devices are connected to the SBC. These are a positioning module and a 4K camera mounted on a gimbal. As mentioned in the previous section, if the air station is to operate in hard-real-time using a Wi-Fi 5 link, then highly reliable, low-latency air-to-ground communication will only be possible with a low link load. This means that the air-to-ground video transmission over a Wi-Fi 5 connection shared with the IoT should be abandoned. The air station can obtain data from analog and digital sensors on board the UAV and from IoT devices outside the UAV. This data can be immediately encapsulated in an MQTT message and

transmitted to the ground or processed on board. The air station can provide AMEC services and process data using artificial intelligence. For air-ground communication, in addition to the Wi-Fi 5 interface, the air station can use external network cards connected to the SBC via the Universal Serial Bus (USB) 3.0 port.

Figure 1 shows the cooperation of the flying IoT system, built on the basis of our framework, with the services of the smart city. The data acquired by the flying IoT system is received by the WMMS. Due to the presence of sensitive data and/or the private data of the monitoring entity in the collected data, the distribution of the full data (red arrows in Fig. 1) is limited to low-latency-enabled services processed on-site. All data can also be sent to a private cloud or to isolated services in a public cloud. The filtered data (green arrows in Fig. 1) is the data to which filters have been applied to remove selected information. This data can be sent directly from WMMS or a private cloud to the smart city cloud. Services that use data from multiple sources, including the flying IoT system, such as analytics services and other applications that do not require low latency, are implemented in the smart city cloud.

WebRTC-based IoT

WebRTC applications are written using the Hypertext Markup Language 5 (HTML5) and the JavaScript programming language. Applications are embedded in web pages, loaded from web servers, and executed using web browsers as runtimes. Due to the need to preserve context, WebRTC applications are written using the One-Page Application (OPA) paradigm. Existing WebRTC use cases, originally published in Request for Comments (RFC) 7478 document[58], did not include the IoT. Only new use cases, presented in WebRTC Extended Use Cases document[51], have announced the need to work on the IoT based on WebRTC. The WebRTC peer-to-peer architecture allows for an easy break from the classic, server-driven concept of the WoT and the building an IoT application that is browser-driven, i.e., works in a typical IoT way[4]. Currently built WebRTC sensor applications are not open systems like the IoT, but rather closed ones. They are built both in a WoT-like way, using a proxy server[59], and in a peer-to-peer architecture[47].

The WebRTC-IoT application is the software of the UAV- and WebRTC-based framework proposed by Chodorek et al.[5], which integrates three parallel services: a sensor service, a positioning service and a video service. These services deal with information captured at the air station. The sensor service is intended to handle sensors connected to the SBC. The sensor data and its metadata, i.e. time and position, are forwarded to the IoT broker that is part of the sensor service, where they are encapsulated in separate MQTT messages and sent to the ground using the WebRTC Data Channel. The positioning service retrieves the current UAV's position from the positioning module, stores it in a buffer for use by the sensor service, and then sends the position to the ground using the WebRTC Data Channel for further use as spatial metadata for the video service. The video service handles a classic WebRTC video. In applications supporting low latency, video transmission is not performed.

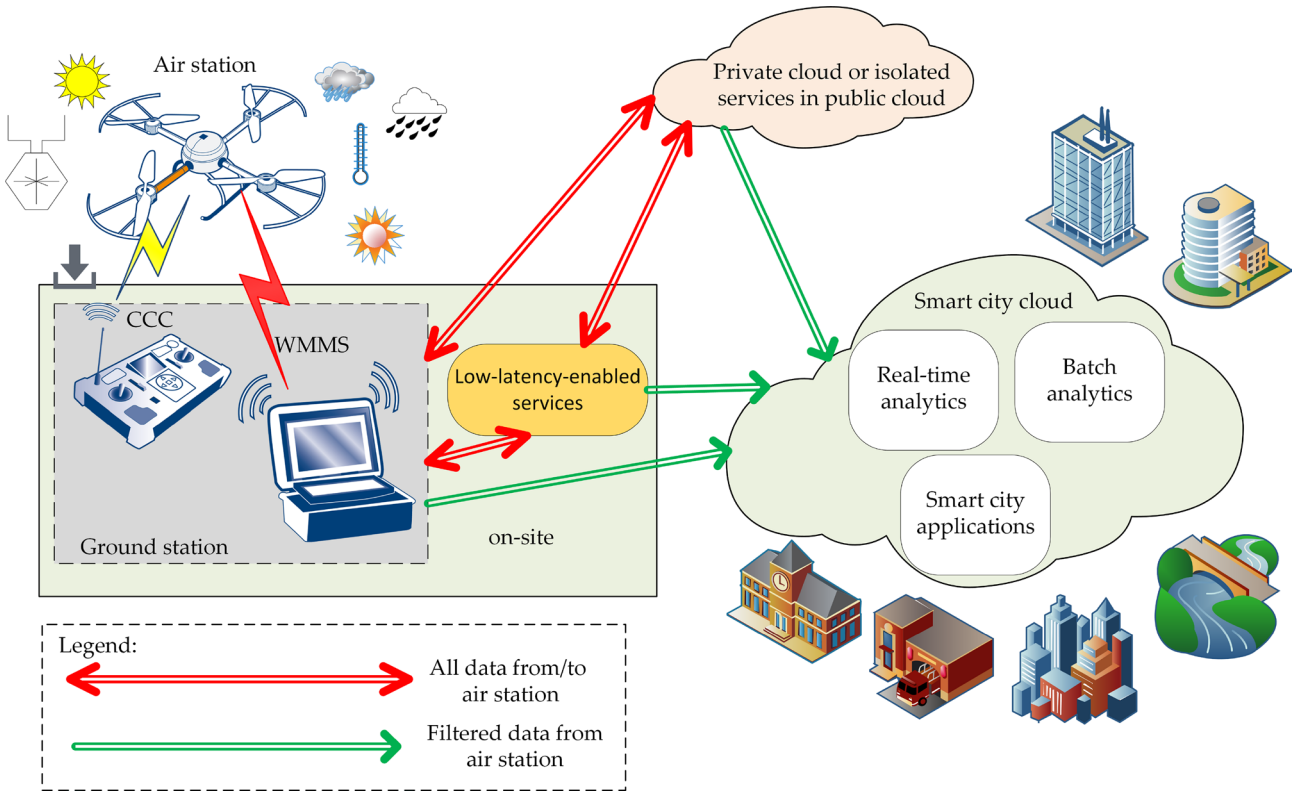


Fig. 1. The concept of low-latency monitoring for smart cities using the UAV- and WebRTC-based universal framework to monitor urban areas.

Part of the sensor service code, including sensor initialization and sensor data reading, is strictly dependent on the sensors used. This part of the code does not belong to the framework[5]. It should be written specifically for a given sensor and integrated with the framework software. Since this paper does not aim to test a specific smart city service, but only the transmission service, the sensors previously used by Chodorek et al.[28,29] are used as sample sensors. This allowed the authors to use ready-made fragments of sensor-dependent code written for the experiments discussed in previous works[28,29]. The WebRTC application running on the air station uses the Chromium browser working in headless mode, i.e. without a graphical user interface (GUI). The SBC on the air station runs under the control of the Raspbian operating system (Linux kernel version 5.4). The WebRTC application running on the ground station uses the Chrome browser, which runs on the Windows operating system.

Reference system

In the reference system, the WebRTC's web logical channel, i.e. Data Channel, was replaced by the classic web logical channel, i.e. WebSocket. As a result, IoT transmissions carried out by both the sensor service and the positioning service were performed via the classic web logical channel, not the WebRTC Data Channel. One of the effects of this was that in the transport layer SCTP was replaced by TCP, and in the session layer DTLS was replaced by TLS. The PUSH option was set in each TCP packet conveying a MQTT message, and the Nagle's algorithm was disabled. The use of reference systems that employ the classic transport protocol, which is TCP, is found in the literature on time-sensitive applications (e.g.[46,56,59]), and WebSocket is used as a reference for WebRTC applications[18,59]. During the experiments, each reference IoT transmission via WebSocket was performed immediately after the completion of the IoT transmission via the WebRTC Data Channel. This ensured that there were no major differences in either the network conditions or other circumstances of the experiment. Performing the reference transmission immediately after the evaluated transmission required the exchange of applications running both on the air and ground stations. The applications were loaded from a web server and launched in browsers during an approximately two-minute maintenance break associated with replacing the UAV's batteries.

Experiments and processing

This section presents fights and flight sessions, the network used in the experiments, time measurements, the selection of a series of end-to-end delays, and the measures of spread used to analyze delay jitter.

Flights, network, and flight sessions

The flights took place on the campus of the AGH University of Krakow, Poland, in a 70 × 70 m parking lot located between the teaching buildings and the dormitory (Fig. 2, Fig. 3). The air station moved at an altitude of 15 m according to the classic geodetic flight scheme, also known as the survey flight pattern, from one corner of the square parking lot to the opposite one, and after reaching the opposite corner, it returned diagonally to the starting point (Fig. 2). During the flight along the longer sections of the flight route, including the diagonal, the movement phase alternated with the hover phase. During each flight, an IoT transmission was carried out

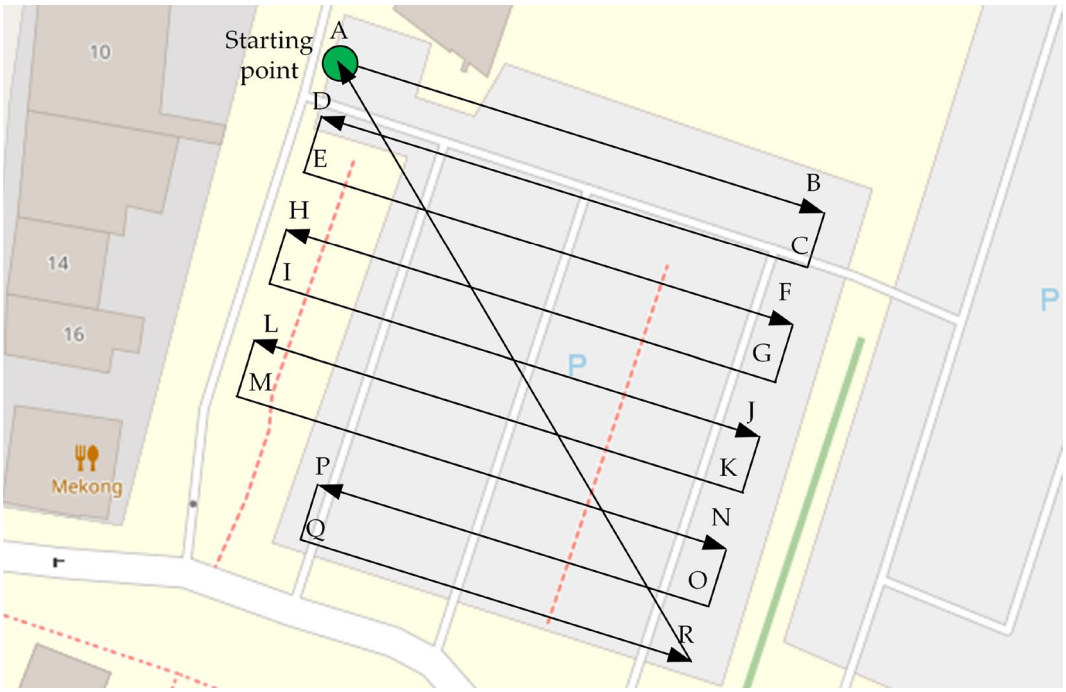


Fig. 2. Trajectory of the air station (background from OpenStreetMap[60]).



Fig. 3. View from a height of 15 m of the parking lot and dormitory.

between the air station and the ground station. After the flight, the batteries of the flight station were replaced if necessary.

The entire parking lot was covered by a dedicated Wi-Fi 5 network with a latency of up to 2 ms, whose access points (APs) were located close to points A, M and N (Fig. 2). During each flight, the transmitter of the air station repeatedly switched between APs. Due to the lack of APs close to point B, during flights near point B in poor network conditions, the transport protocol detected single transmission errors (one per flight). The APs were connected to a Gigabit Ethernet switch, creating a wired Wi-Fi Extended Service Set's (ESS) Distributed System (DS). As a result, the transmission between the air station and the ground station was either carried out exclusively via the wireless network (via an AP located near point A) or via a heterogeneous wired-wireless network (via an AP located near point M or N, then via a switch and via an AP located near point A). The APs were NETGEAR Nighthawk X4 R7500 AC2350 and the switch is HP 3500-24G-PoE+ yl.

The flights were organized in flight sessions, which took place in different weather conditions and with different loads on the coexisting Wi-Fi networks. In each flight session, 5 pairs of flights were performed. During the first flight, the IoT data were transmitted via WebRTC (evaluated transmission), and during the second, the IoT data were transmitted via WebSocket (reference transmission). Just before and just after each flight session, the clocks of the flight station and the ground station were synchronized. The synchronization performed after the flight session was aimed at detecting time drift. In total, 8 flight sessions were conducted, from the end of January to the end of May, each consisting of 5 pairs of flights, giving 40 transmission pairs in total. In the case of one of the flight sessions, the time synchronization performed after the flight session showed a time drift. All pairs of transmissions carried out during this flight session were rejected, and the delays collected at that time were not subject to statistical processing.

Selecting pairs of evaluated and reference transmissions

As mentioned above, out of 40 transmission pairs (an IoT over WebRTC transmissions and the immediately following IoT over WebSocket transmissions), 5 were rejected due to detected time drift. Out of the remaining 35 pairs, five were selected for further analysis. The main selection criterion was the network operating conditions. Chodorek and Chodorek[18] stated that a good indicator of the network operating conditions is the number of errors detected in the medium access control (MAC) sublayer. The better the network operating conditions, the lower the number of errors detected in the MAC sublayer. Since about half of all transmissions were carried out under good network conditions, one-fourth under medium network conditions, and one-fourth under poor network conditions, representatives of transmission pairs were selected in proportion to the share of transmissions performed under each condition. Two pairs were selected from those made under good network conditions (day 1 and day 3), and one pair each was selected from those made under medium and poor network conditions (day 5 and day 4, respectively). The one and only pair, in which the evaluation transmission was error-free at the transport layer, but during the comparative transmission the TCP detected an error, was also selected (day 2). The five selected pairs of transmissions were conducted on different days, at different times of the day (morning flight session, midday one, and evening one), and in different weather conditions, i.e. from cool to warm days (1.5 to 25 degrees Celsius), during dry weather, rainy weather and just after rain (relative humidities from 46% to 85%).

End-to-End delays

Every half a second, the air station sent a burst of nine data coming from five environmental sensors (one analog and four digital) and their metadata (time and position). For each set of IoT data sent and each set of IoT metadata sent, a number of time parameters were collected, both at the air and ground stations. These were:

- time of entering a logical channel t_i^{ilc} by the *i*-th datum,
- time of receiving the *i*-th datum from a transport protocol t_i^t,
- time of receiving the *i*-th datum from a logical channel t_i^{olc}.

The difference between the time of entering the logical channel and the time of receiving the *i*-th datum from a transport protocol is the end-to-end delay d_i^t measured at the level of the transport protocol, or more succinctly, at the transport level (Fig. 4):

$$d_i^t = t_i^t - t_i^{ilc} \tag{1}$$

The difference between the time of entering the logical channel and the time of receiving the *i*-th datum from the logical channel is the end-to-end delay d_i^{lc} measured at the level of the logical channel (Fig. 4):

$$d_i^{lc} = t_i^{olc} - t_i^{ilc} \tag{2}$$

For each evaluated transmission (i.e. WebRTC one, subscript WRTC), a D_{WRTC}^t series of delays $d_i^t, i = 1, 2, ..., N$, i.e. measured at the transport level, and a D_{WRTC}^{lc} series of delays $d_i^{lc}, i = 1, 2, ..., N$, i.e. measured at the logical channel level, were created. For each reference transmission (i.e. WebSocket one, subscript WS), a D_{WS}^{lc} series of delays $d_i^{lc}, i = 1, 2, ..., N$, i.e. measured at the logical channel level, were created. The lack of the D_{WS}^t series is due to the fact that WebSocket does not allow for time measurement at the transport protocol level. The value of *N* was set to 40,000, which is due to the general requirement of high reliability in low-latency systems (Table 1). The authors assumed a reliability constraint of 99.99% to 99.999%. The value of *N* of 40,000 provided a delivery rate of 99.9975% in the event of a single transmission error at the transport layer level. The series were spatially synchronized. The index $i = 1$ means that the air station was directly above the ground station.

The series were statistically processed to evaluate the fluctuations of the end-to-end delays measured at the transport level and at the logical channel level. The evaluation was performed using classical measures of spread: range (*R*), interquartile range (*IQR*), variance (*Var*), standard deviation (σ), and coefficient of variation (*CoV*). These measures were calculated as follows:

$$R(D) = max(D) - min(D) \tag{3}$$

where *D* is a given series of end-to-end delays, $D \in \{D_{WRTC}^t, D_{WRTC}^{lc}, D_{WS}^{lc}\}$, *max*(*D*) is the maximum end-to-end delay in the given series, and *min*(*D*) is the minimum end-to-end delay in the given series.

$$IQR(D) = Q3(D) - Q1(D) \tag{4}$$

where *Q*3(*D*) is the upper quartile of a given series and *Q*1(*D*) is the lower quartile of a given series.

$$Var(D) = \frac{1}{card(D)} \sum_{i=1}^{card(D)} (d_i - \mu(D))^2, d_i \in D \tag{5}$$

where $\mu(D)$ is the arithmetic mean of the given series.

$$\sigma(D) = \sqrt{Var(D)} \tag{6}$$

$$CoV(D) = \frac{\sigma(D)}{\mu(D)} \tag{7}$$

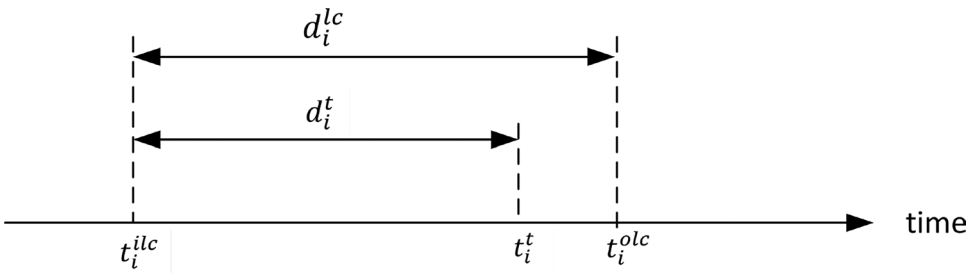


Fig. 4. Time parameters and time intervals.

Days	$R(D^t_{WRTC})$	$IQR(D^t_{WRTC})$	$Var(D^t_{WRTC})$	$\sigma(D^t_{WRTC})$	$CoV(D^t_{WRTC})$
day 1	74 μs	10 μs	167	12.9	0.373%
day 2	85 μs	14 μs	212	14.6	0.421%
day 3	65 μs	13 μs	139	11.8	0.341%
day 4	7,939 μs	10 μs	145,385	381.3	10.92%
day 5	92 μs	11 μs	201	14.2	0.409%

Table 2. Measures of spread (range, interquartile range, variance, standard deviation, and coefficient of variation) of end-to-end delays measured at the transport level when the IoT data were transmitted through the WebRTC data channel.

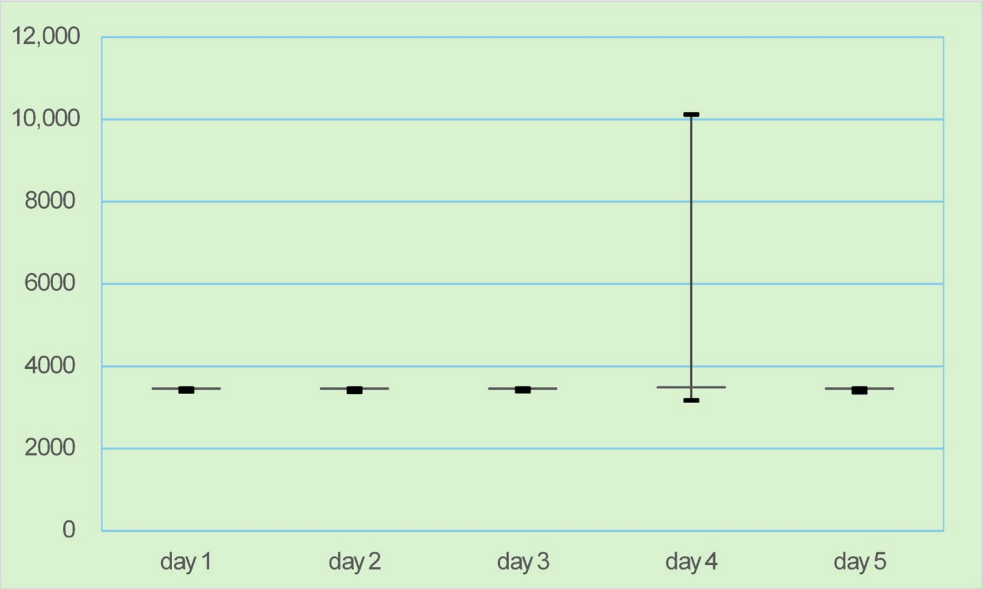


Fig. 5. The range and the interquartile range of the end-to-end delays measured at the transport level when the IoT data were transmitted over WebRTC.

For the sake of the evaluation of the impact of single outliers on measures of spread, especially the impact of the delay of IoT datum conveyed in the retransmitted packet, truncated statistics were also calculated. These were: truncated range, truncated interquartile range, truncated variance, truncated standard deviation, and truncated coefficient of variation. The truncated measures of spread were calculated from the truncated series (*DT*), i.e. the *D* series of end-to-end delays from which the two extreme delays (minimum and maximum delays) were discarded.

Results

This section presents and discusses the computational results of measures of spread of end-to-end delays of WebRTC-based IoT traffic, performed at both the transport protocol level and at the web logical channel level. To better demonstrate the impact of the retransmitted packet on the end-to-end delay measures of spread computed at each of these levels, truncated measures of spread are also presented and discussed.

Measures of spread at the transport level

The MQTT message decapsulated from the SCTP packet is passed from the transport protocol to the buffer of the web logical channel. The time of receiving this message from SCTP is the basis for estimating the end-to-end delay at the transport protocol level, expressed by (1). Because the WebRTC's transport protocol does not reorder misordered packets, the raw flow of received SCTP payloads is forwarded to the buffer unchanged, as is, without waiting for a delayed or lost packet to arrive. Measures of the spread of a series of end-to-end delays D^t_{WRTC} obtained at the transport level on days 1 to 5 are summarized in Table 2. This table includes range $R(D^t_{WRTC})$, interquartile range $IQR(D^t_{WRTC})$, variance $Var(D^t_{WRTC})$, standard deviation $\sigma(D^t_{WRTC})$, and coefficient of variation $CoV(D^t_{WRTC})$ of the delays measured when the IoT data embedded in MQTT messages were transmitted over the WebRTC Data Channel.

In the case of the four experiments conducted on days 1-3 and day 5, during which no transmission errors were detected in the transport layer ($PER = 0$), the range of end-to-end delays (Table 2, Fig. 5) was from 65 µs (day 3) to 92 µs (day 5), and the interquartile range was from 10 µs (day 1) to 14 µs (day 3). This two-digit

Days	$R(D^{lc}_{WRTC})$	$IQR(D^{lc}_{WRTC})$	$Var(D^{lc}_{WRTC})$	$\sigma(D^{lc}_{WRTC})$	$CoV(D^{lc}_{WRTC})$
day 1	74 μs	10 μs	167	12.9	0.373%
day 2	85 μs	14 μs	212	14.6	0.421%
day 3	65 μs	13 μs	139	11.8	0.341%
day 4	7,939 μs	10 μs	186,637	432	12.36%
day 5	92 μs	11 μs	201	14.2	0.409%

Table 3. Measures of spread of end-to-end delay measured at the logical channel level when the IoT data were transmitted over the WebRTC.

Days	$\frac{R(D^{lc}_{WRTC})}{R(D^{t}_{WRTC})}$	$\frac{IQR(D^{lc}_{WRTC})}{IQR(D^{t}_{WRTC})}$	$\frac{Var(D^{lc}_{WRTC})}{Var(D^{t}_{WRTC})}$	$\frac{\sigma(D^{lc}_{WRTC})}{\sigma(D^{t}_{WRTC})}$	$\frac{CoV(D^{lc}_{WRTC})}{CoV(D^{t}_{WRTC})}$
day 1	1	1	1	1	1
day 2	1	1	1	1	1
day 3	1	1	1	1	1
day 4	1	1	1.28	1.13	1.13
day 5	1	1	1	1	1

Table 4. Comparison of measures of spread of the end-to-end delays measured at the logical channel level and the transport level when the IoT data were transmitted using WebRTC Data Channel.

microsecond jitter can be considered zero millisecond jitter, to the one tenth of a microsecond (range) or one hundredth of a microsecond (interquartile range). During error-free transmission, the values of other measures of spread are also small. The variance is less than or equal to 212 μs^2. The standard deviation is greater than ten microseconds (from 12 in day 3 to 15 in day 2). The coefficient of variation is less than one percent, indicating a strong concentration of the instantaneous end-to-end delay values around the arithmetic mean.

On day 4, a single transmission error was detected in the transport layer, which increased the PER to 0.0025%. As a result, a retransmission of the SCTP packet occurred, which caused an increase in the measures of spread compared to the measures calculated for error-free transmissions (Table 2, Fig. 5). As an effect, measures calculated for day 4 were an order of magnitude (standard deviation), two orders of magnitude (range and coefficient of variation) and three orders of magnitude (variance) higher than the same measures calculated for days 1 to 3 and 5. Although a coefficient of variation of less than 20% is usually interpreted as low variation, in the case of measurements carried out on day 4 the jitter cannot be considered zero. The only measure of spread that has not increased is interquartile range, insensitive to extreme values. Moreover, the interquartile ranges calculated for day 4 and day 1 takes the best (lowest) value of all interquartile ranges given in Table 2.

Measures of spread at the logical channel level

As mentioned in the previous section, in the case of WebRTC, the SCTP protocol does not reorder packets that are received out of order. After decapsulation, the data is immediately transferred to the buffer. If the WebRTC Data Channel's configuration requires passing data to the application in the same order in which they were sent, and the data arrives in the wrong order, the order of the received data is changed at the logical channel level, using WebRTC Data Channel procedures. Table 3 presents measures of spread obtained at the logical channel level, for IoT transmissions conducted over the WebRTC Data Channel. They are: range $R(D^{lc}_{WRTC})$, interquartile range IQR $IQR(D^{lc}_{WRTC})$, variance $Var(D^{lc}_{WRTC})$, standard deviation $\sigma(D^{lc}_{WRTC})$, and coefficient of variation $CoV(D^{lc}_{WRTC})$. Table 4 compares the measures of spread calculated for series collected at the level of logical channel (Table 3) and at the level of transport protocol (Table 2).

For error-free transmissions, which occurred on days 1-3 and 5, SCTP packets did not require reordering. As an effect, the values of the measures of spread calculated from the series collected on these days at the logical channel level, given in Table 3 and Fig. 6, are exactly the same as the values of the measures of spread calculated from the series collected at the transport protocol level, given in Table 2 and Fig. 5. The equality of the measures of spread is shown in Table 4, where for days 1 to 3 and 5 the quotients of the corresponding measures are 1. While at the transport level, the measures of spread listed in Table 2 for $PER = 0$ were equal due to the same end-to-end delays, at the logical channel level the equality of the measures of spread listed in Table 2 and Table 3 is due to the same buffering delays d^b_i. If the WebRTC transmission was error-free, the end-to-end delay measured at the logical channel level was an affine transformation of the end-to-end delay measured at the transport protocol level:

$$d^{lc}_i = d^t_i + d^b_i, d^b_i = d^b_{WRTC} = const., i = 1, 2, ..., \ 40,000, \ \text{if } PER = 0 \tag{8}$$

where d^b_{WRTC} is the minimum buffering delay introduced by processing in the WebRTC Data Channel buffer. Chodorek and Chodorek[18] showed that if the PER in the transport layer was zero, the processing in the buffer

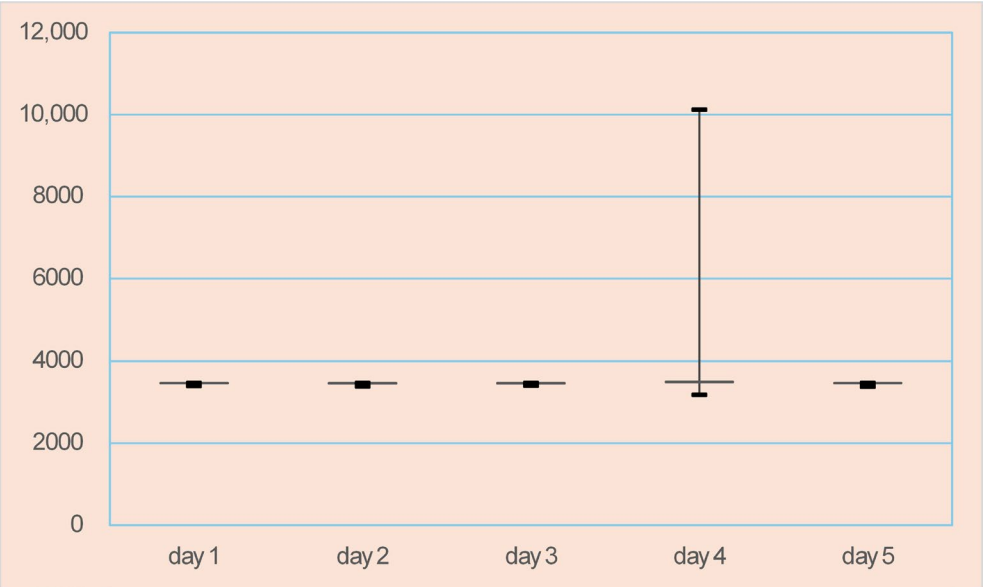


Fig. 6. The range and the interquartile range of the end-to-end delays measured at the logical channel level when the IoT data were transmitted over WebRTC.

increased the values of the analyzed measures of location (arithmetic mean, median, mode, upper quartile, and lower quartile) and the extreme values by 2 μs. It can therefore be concluded that:

$$d^{b}_{WRTC} = 2\ \mu s,\ \text{if}\ PER = 0 \tag{9}$$

Thus,

$$d^{lc}_{i} = d^{t}_{i} + 2\ \mu s, i = 1, 2, ...,\ 40,000,\ \text{if}\ PER = 0 \tag{10}$$

On day 4, the transport protocol detected a single transmission error and retransmitted one lost packet. This caused packets sent after the lost packet and received before the arrival of the retransmitted lost packet to be misordered. Their payloads were transferred to the buffer, where they waited to be re-ordered and reported ready to the application. This was only possible after the lost packet was retransmitted and its payload passed to the buffer. The delays of the payloads waiting in the buffer were lower than the delay of the retransmitted packet payload, so they were not the basis for calculating the range. Due to the highly reliable, low-latency underlying network, their percentage of the total number of payloads was too small to affect the interquartile range. As a result, the end-to-end delays used to calculate the range and the interquartile range underwent the transformation given by (9). The range and interquartile range values calculated at the transport level (Table 2) are equal to the values of the same measures calculated at the logical channel level (Table 3), and this equality makes the quotient value shown in Table 4 equal to 1. This does not mean, however, that transformation (9) occurs for all delays.

A quotient greater than 1, indicating greater dispersion of end-to-end delays measured at the logical channel level, occurred for variance and two variance-related measures. As Table 4 shows, the variance calculated at the logical channel level increased by almost 30% compared to the variance calculated at the transport level, i.e. from 145,385 (Table 2) to 186,637 (Table 3). This resulted in an increase in the standard deviation by over 10%, from 381.3 μs (Table 2) to 432 μs (Table 3), and in the coefficient of variation also by over 10%, from 10.92% (Table 2) to 12.36% (Table 3). The same quotient values in the last two columns of Table 4, calculated for day 4, result from approximately the same, with an accuracy of d^{b}_{WRTC}, arithmetic means calculated at the transport level and the logical channel level. Chodorek and Chodorek[18] reported that, on day 4, buffer processing increased the mean, median, mode, upper quartile, and lower quartile values by 2 μs as well.

Measures of spread of truncated series

Discarding the extreme delays from the end-to-end delay series creates a truncated series. In our considerations, truncated series allow for the analysis of the measures of spread of the delay series, without taking into account the delay of the retransmitted packet. Table 5 show the measures of spread of the truncated series of end-to-end delays measured at the transport level during the IoT data transmission through the WebRTC Data Channel. The table summarizes: truncated range $R(DT^{t}_{WRTC})$, truncated interquartile range $IQR(DT^{t}_{WRTC})$, truncated variance $Var(DT^{t}_{WRTC})$, truncated standard deviation $\sigma(DT^{t}_{WRTC})$, and truncated coefficient of variation $CoV(DT^{t}_{WRTC})$. Table 6 compares the measures of spread calculated for full series (Table 2) and truncated one (Table 5). The comparison was made in the form of a quotient of the values of a given statistical measure calculated for the full and truncated series.

Days	$R(DT^t_{WRTC})$	$IQR(DT^t_{WRTC})$	$Var(DT^t_{WRTC})$	$\sigma(DT^t_{WRTC})$	$CoV(DT^t_{WRTC})$
day 1	74 μs	10 μs	167	12.9	0.373%
day 2	85 μs	14 μs	212	14.6	0.421%
day 3	65 μs	13 μs	139	11.8	0.341%
day 4	316 μs	10 μs	867	29.4	0.843%
day 5	92 μs	11 μs	201	14.2	0.409%

Table 5. Truncated measures of spread of end-to-end delays measured at the transport level when the IoT data were transmitted through the WebRTC Data Channel.

Days	$\frac{R(D^t_{WRTC})}{R(DT^t_{WRTC})}$	$\frac{IQR(D^t_{WRTC})}{IQR(DT^t_{WRTC})}$	$\frac{Var(D^t_{WRTC})}{Var(DT^t_{WRTC})}$	$\frac{\sigma(D^t_{WRTC})}{\sigma(DT^t_{WRTC})}$	$\frac{CoV(D^t_{WRTC})}{CoV(DT^t_{WRTC})}$
day 1	1	1	1	1	1
day 2	1	1	1	1	1
day 3	1	1	1	1	1
day 4	25.12	1	167.69	12.95	12.95
day 5	1	1	1	1	1

Table 6. Comparison of measures of spread and corresponding truncated measures of spread of the end-to-end delays measured at the transport level when the IoT data were transmitted using WebRTC Data Channel.

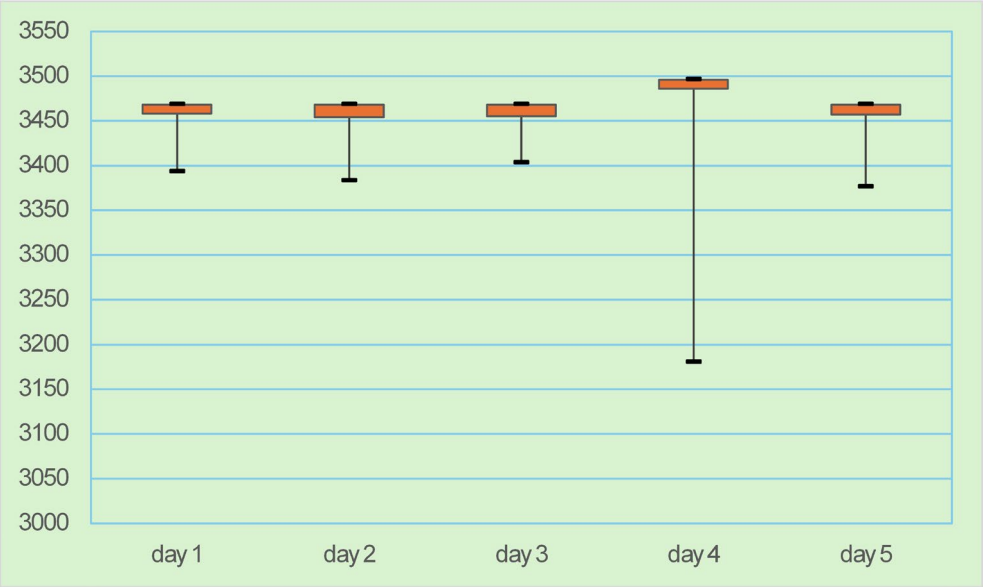


Fig. 7. The truncated range and the truncated interquartile range of the end-to-end delays measured at the transport level when the IoT data were transmitted over WebRTC.

As shown in Table 5, discarding the extreme delays from the series of end-to-end delays collected during error-free transmissions (days 1 to 3 and 5) does not change the measures of spread. The quotient of 1, appearing in Table 6 for days 1-3 and 5, showed that on these days the measures of spread calculated from the full series were equal to the corresponding measures calculated from the truncated series. This proves that although IoT traffic is very bursty and the test environment is characterized by high variability in network conditions, in the case of error-free transmission, extreme end-to-end transmission delays are not outliers. If the transmission is error-free, the extreme end-to-end transmission delays are not outliers but belong to a set of similar values. Their removal did not change either the range of the series or the spread of delays around the arithmetic mean.

Unlike the truncated series from days 1-3 and 5, discarding the extremes from the series collected on day 4 resulted in a significant reduction in the maximum value because a single outlier was then removed (Fig. 5, Fig. 7), i.e. the very high delay of the retransmitted packet. Discarding this delay from the series entailed the significant reduction in the measures of spread, as shown in Table 5 and Fig. 7. Truncated measures are an order of magnitude smaller (truncated range, truncated standard deviation, and truncated coefficient of variation)

Days	$R(DT^{lc}_{WRTC})$	$IQR(DT^{lc}_{WRTC})$	$Var(DT^{lc}_{WRTC})$	$\sigma(DT^{lc}_{WRTC})$	$CoV(DT^{lc}_{WRTC})$
day 1	74 μs	10 μs	167	12.9	0.373%
day 2	85 μs	14 μs	212	14.6	0.421%
day 3	65 μs	13 μs	139	11.8	0.341%
day 4	7,004 μs	10 μs	101,186	318.1	9.104%
day 5	92 μs	11 μs	201	14.2	0.409%

Table 7. Truncated measures of spread of end-to-end delays measured at the channel level when the IoT data were transmitted through the WebRTC Data Channel.

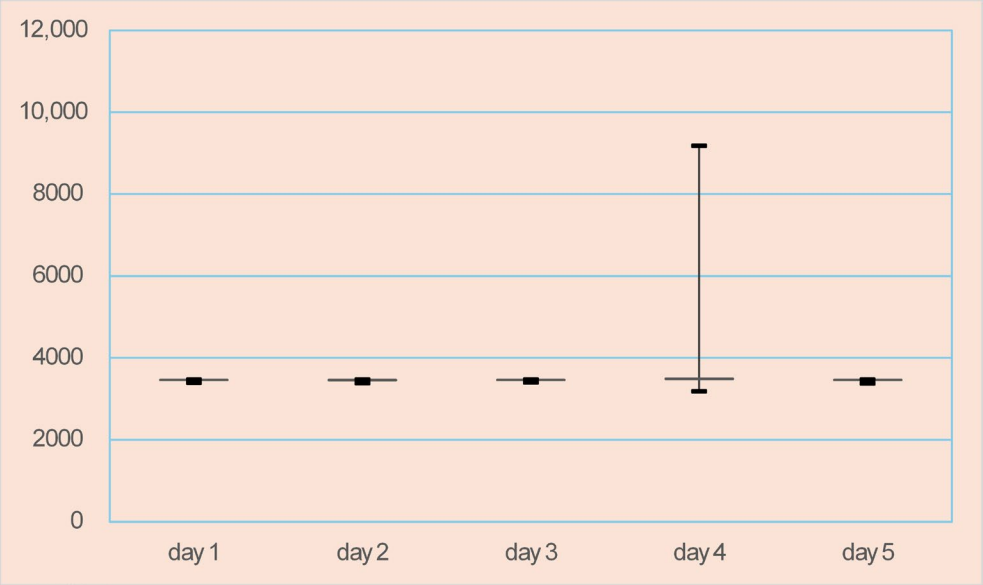


Fig. 8. The truncated range and the truncated interquartile range of the end-to-end delays measured at the logical channel level when the IoT data were transmitted over WebRTC.

or three orders of magnitude smaller (truncated variance) than values calculated from the full series. Only the interquartile range, insensitive to outliers, calculated from the full series is equal to that calculated from the truncated series. This is shown as a quotient of 1 in Table 6. The quotient greater than 1, appearing on day 4, shows how many times the measures of spread calculated from the full series are greater than the same measures calculated from the truncated series. As shown in Table 6, they are from about ten times (standard deviation and coefficient of variation), through 25 times (range) to about 170 times (variance) larger. However, these values are still several times higher than the values of the corresponding statistics obtained on days 1-3 and 5, when transmissions were error-free.

The measures of spread of the truncated end-to-end delay series measured at the logical channel level during the transmission of IoT data over the WebRTC Data Channel are presented in Table 7. The table summarizes: truncated range $R(DT^{lc}_{WRTC})$, truncated interquartile range $IQR(DT^{lc}_{WRTC})$, truncated variance $Var(DT^{lc}_{WRTC})$, truncated standard deviation $\sigma(DT^{lc}_{WRTC})$, and truncated coefficient of variation $CoV(DT^{lc}_{WRTC})$. The first two measures, i.e. the truncated range and the truncated interquartile range, are also illustrated in Fig. 8. The quotient of the measures of spread calculated for full series (Table 3) and truncated series (Table 7) is presented in Table 8.

The results presented in Fig. 8 and Table 7 show that discarding the extremes from the end-to-end delay series did not result in such a spectacular reduction in measures of spread as was the case with the transport level. In the case of error-free transmissions (day 1-3 and day 5), there are no differences between the corresponding measures of spread calculated for full and truncated series. In the light of the considerations presented so far, this is a typical result for the analyzed system. When an error was detected in the transport layer (day 4), due to the long waiting times for reordering, payloads waiting in the buffer significantly influenced the statistics of end-to-end delays measured at the level of the logical channel. Therefore, the truncated measures of spread calculated at the logical channel level are larger than those calculated at the transport level, except for the interquartile range. On day 4, the quotient of the measures of spread presented in Table 8 is much smaller than the quotient calculated for the transport level and at most exceeds 1.8 (variance). The other two variance-based measures, calculated from the full series, are 1/3 smaller than those calculated from the truncated ones, and the range is just over 10% larger.

Days	$\frac{R(D^{lc}_{WRTC})}{R(DT^{lc}_{WRTC})}$	$\frac{IQR(D^{lc}_{WRTC})}{IQR(DT^{lc}_{WRTC})}$	$\frac{Var(D^{lc}_{WRTC})}{Var(DT^{lc}_{WRTC})}$	$\frac{\sigma(D^{lc}_{WRTC})}{\sigma(DT^{t}_{WRTC})}$	$\frac{CoV(D^{lc}_{WRTC})}{CoV(DT^{lc}_{WRTC})}$
day 1	1	1	1	1	1
day 2	1	1	1	1	1
day 3	1	1	1	1	1
day 4	1.13	1	1.84	1.36	1.36
day 5	1	1	1	1	1

Table 8. Comparison of measures of spread and corresponding truncated measures of spread of the end-to-end delays measured at the channel level when the IoT data were transmitted using WebRTC Data Channel.

Days	$R(D^{lc}_{WS})$	$IQR(D^{lc}_{WS})$	$Var(D^{lc}_{WS})$	$\sigma(D^{lc}_{WS})$	$CoV(D^{lc}_{WS})$
day 1	67,315 μs	3,816 μs	28,492,251	5,338	67.17%
day 2	81,132 μs	21,724 μs	60,118,650	7,754	44.04%
day 3	63,178 μs	2,516 μs	12,951,023	3,599	49.57%
day 4	84,630 μs	23,615 μs	319,113,212	17,864	79.10%
day 5	72,184 μs	14,852 μs	33,399,250	5,779	44.35%

Table 9. Measures of spread (range, IQR, variance, standard deviation, and coefficient of variation) of end-to-end delay measured at the logical channel level when the IoT data were transmitted over the WebSocket.

Days	$\frac{R(D^{lc}_{WS})}{R(D^{lc}_{WRTC})}$	$\frac{IQR(D^{lc}_{WS})}{IQR(D^{lc}_{WRTC})}$	$\frac{Var(D^{lc}_{WS})}{Var(D^{lc}_{WRTC})}$	$\frac{\sigma(D^{lc}_{WS})}{\sigma(D^{lc}_{WRTC})}$	$\frac{CoV(D^{lc}_{WS})}{CoV(D^{lc}_{WRTC})}$
day 1	910	382	170,612	413	180
day 2	955	1,552	283,579	533	105
day 3	972	194	93,173	305	146
day 4	10.7	2,362	1,710	41.4	6.4
day 5	785	1,350	166,165	408	108

Table 10. Comparison of measures of spread calculated from end-to-end delays collected at the logical channel level when IoT data was transmitted via Websocket (D^{lc}_{WS} series) and via WebRTC Data Channel (D^{lc}_{WRTC} series).

Comparison

Measures of spread: the range $R(D^{lc}_{WS})$, the interquartile range $IQR(D^{lc}_{WS})$, the variance $Var(D^{lc}_{WS})$, the standard deviation $\sigma(D^{lc}_{WS})$, and the coefficient of variation $CoV(D^{lc}_{WS})$, calculated from the series of end-to-end delays measured at the level of logical channel during reference IoT transmissions are summarized in Table 9. Even a cursory look at Table 9 shows that changing the web logical channel used for IoT transmission from WebRTC Data Channel to WebSocket resulted in very large values of the measures of spread, regardless of whether the transmission was error-free (odd days) or not (even days). These are such large values that not only the zero jitter requirement, but also the low jitter requirement, cannot in any way be considered to be met, even when the PER is zero, and even to a very close approximation. In detail, the range is from 63.2 ms to 84.6 ms, the interquartile range is from 2.5 ms to 23.6 ms, the variance is from 12.95 ms^2 to 319.11 ns^2, and the standard deviation is from 3.6 ms to 17.9 ms. This is from an order of magnitude (range at day 4) to five orders of magnitude (variance at days 1 to 3 and at day 5) larger than the corresponding values shown in Table 3, calculated from the D^{lc}_{WRTC} series.

Coefficients of variation range from 40% to 100%, which is typically interpreted as high variation. While the remaining measures of spread calculated when PER was non-zero are more or less significantly higher than those calculated for zero PER, the non-zero PER gave both the highest calculated coefficient of variation (79%, day 4) and the lowest (44%, day 2). In comparison, when the WebRTC data channel was used, non-zero PER resulted in an increase in the coefficient of variation by two orders of magnitude (Table 3). Moreover, except the interquartile range, transmission with non-zero PER always clearly increased the measures of spread, by at least one order of magnitude. The interquartile range, being insensitive to outliers, did not increase with non-zero PER when the WebRTC logical channel was used. When the WebSocket logical channel was used under the same circumstances, the interquartile range calculated for non-zero PER increased significantly.

Table 10 compares the measures of spread calculated from the series collected during IoT transmissions over the classic web logical channel, i.e. WebSocket (Table 9) and the WebRTC logical channel, i.e. Data Channel (Table 3). The quotients in Table 10 are always greater than 1. Thus, each measure showed a larger spread in end-

to-end delays when the WebSocket was used, and a smaller spread when the WebRTC Data Channel was used. The largest quotients occur for days 1-3 and 5, when IoT transmissions via WebRTC were error-free. The spread of end-to-end delay values collected during transmission via WebSocket is then, depending on the measure, from over a hundred times (coefficient of variation) to hundreds of thousands of times (variance) greater than when using WebRTC Data Channel under the same conditions. The differences on day 4, when both WebRTC and WebSocket transmissions experienced one transmission error, are smaller but still significant.

As was mentioned above, the QUIC protocol is considered as a possible successor of TCP in WebSocket. Jeddou et al.[46] showed that in the case of error-free MQTT over QUIC transmission between stationary end systems connected via emulated Wi-Fi, the range between the 5th and 95th percentiles was about 3 ms, and the interquartile range was about 1 ms. For comparison, in our tests, during error-free MQTT over WebRTC transmission between a mobile air station and a stationary ground station, the range between the maximum and the minimum was less than 0.1 ms, and the interquartile range was less than 0.015 ms. Jeddou et al.[46] also reported that the relative standard deviation (RSD), which is equivalent to the coefficient of variation, was about 0.06 (CoV of 6%), and during our test it was less than 0.00421 (CoV of 0.421%). The measures of spread given in Jeddou et al.[46] were read from the plots. Concluding, it can be assessed that in terms of measures of the spread of end-to-end delays, the use of the WebRTC logical channel also gives better results than the use of the QUIC protocol.

Comparing the analyzed system with other WebRTC-based systems known from the literature, it can be stated that systems tested in real networks are typically not tested under conditions of high transmission reliability. In this respect, this paper is therefore a pioneering work. Moreover, the application of WebRTC in IoT systems often comes down to using only the RTP/UDP protocol stack for audio-video transmission, while sensor data are transmitted using a non-WebRTC-based IoT, and systems in which sensor data are transmitted using WebRTC are usually closed ones, although built in a way similar to IoT (browser-driven) or WoT (server-driven). Tiberkak et al.[59] presented WebRTC-based WoT-like remote control of robots. A remote controlled robot with a vision system communicates with the outside world in a manner similar to WoT, i.e., through an intermediary proxy server. The communication from the server to the user is performed using WebRTC. In contrast to our experiments, in the experiments conducted by Tiberkak et al.[59] the clocks of the end systems were not synchronized and the system was evaluated using the round-trip delay. Due to the use of the intermediate server, the estimated end-to-end delays are much larger than in our system. Although Tiberkak et al.[59] does not report the delay jitter, the large differences between the maximum and average round-trip delays show that the delay fluctuations are also much larger.

Kilic et al.[57] presented a UAV-based system for automated power line inspection (APOLI). The system streams video from multiple UAVs in an environment that does not provide either high reliability (the lowest packet error rates of 0.2, i.e., packet delivery rates of 0.8% in 5G) or low latency. The achieved jitter ranged from 1 ms for SD video (bitrate 2 Mbps) transmitted over 5G to 10 ms for Full HD video (bitrate 8 Mbps) transmitted over Wi-Fi. The RTP protocol does not perform retransmission. In our work, if the transmission was not error-free, the jitter expressed in the range between the maximum and minimum values was 7.9 ms in the low-latency and high-reliability Wi-Fi network when taking into account the retransmitted packet (Table 3), and 0.3 ms without taking into account the effect of retransmission (Table 5). There is therefore no need to change the protocol stack used to the RTP/UDP stack.

Black et al.[47] showed a teleoperation system based on WebRTC. It is a system designed for telemedicine, in which video and data are transmitted via WebRTC, but the signaling for WebRTC is transmitted via WebSocket. The system operates in an unreliable environment, but the paper does not contain either packet error rates or packet delivery rates. The network environment does not provide low latency. In the experiments conducted by Black et al.[47], the clocks were not synchronized and the round-trip time was measured. Black et al.[47] reported that the system has a round-trip delay of 5.80 ± 3.30 ms over Wi-Fi. The obtained delay fluctuations are therefore much larger than in our work, which is probably due to the failure to meet the requirements of high transmission reliability. Black et al.[47] concluded that communication should be performed without retransmissions and likely without ordering guarantees. This is also confirmed in this paper, as the absence of retransmission and ordering guarantees reduced the end-to-end delay jitter from almost 7.9 ms to 0.3 ms. However, it should be emphasized that not all IoT systems allow abandoning retransmission and ordering guarantees.

Conclusions

The aim of this paper was to analyze the fluctuations of end-to-end delay measured at the transport protocol level and at the web logical channel level when an IoT on board a UAV transmitted sensor data to the ground using the WebRTC Data Channel. The end-to-end delay fluctuation tests were conducted using a low-latency Wi-Fi 5 network. The variation of network operating conditions was achieved by rapidly changing factors, including the change of the UAV's distance from the ground station or access point, the change of the access point, the use of wireless-only or mixed wired-wireless Wi-Fi distribution system, as well as slowly changing factors such as different weather conditions and different loads of the coexisting networks. The transmission reliability was assumed to be between 99.99% and 99.999% (a single transmission error in 40,000 data packets). As measures of fluctuations in end-to-end delays, five measures of spread were used.

The results show that error-free transmissions achieve very low end-to-end latency jitter, which meets the requirements of nondeterministic low-latency services by a large margin. The jitter achieved at the transport level was then the same as achieved at the logical channel level. If the transport protocol detected a single transmission error, the values of almost all measures of spread, except for the interquartile range, increased significantly. The increase at the logical channel level was much larger than at the logical channel level, due to the data reordering performed by the logical channel. Therefore, the authors recommend not to unnecessarily set the in-order data delivery option in the WebRTC Data Channel configuration. A comparison with reference

transmissions performed using the classic web logical channel, which is Websocket, showed a large superiority of the WebRTC Data Channel in terms of measures of spread of end-to-end delays.

Future research will focus on analyzing WebRTC-based UAV-borne IoT transmissions performed with the use of other communication solutions, such as Wi-Fi 6e and Wi-Fi 7 networks, and 5G network, as well as analyzing a system composed of multiple drones communicating using a 5G test network.

Data availability

The datasets used and/or analysed during the current study available from the corresponding author on reasonable request.

Received: 23 April 2025; Accepted: 20 February 2026
Published online: 26 February 2026

References

1. Wikarek, J., Sitek, P. & Zawarczyński, Ł. An Integer Programming Model for the Capacitated Vehicle Routing Problem with Drones. In: *Computational Collective Intelligence*. ICCCI 2019. Lecture Notes in Computer Science, Vol. 11683, 511–520 (eds Nguyen, N. et al.) https://doi.org/10.1007/978-3-030-28377-3_42 ((Springer: Cham, 2019).
2. Sitek, P., Wikarek, J. & Rutczyńska-Wdowiak, K. Capacitated Vehicle Routing Problem with Pick-Up, Alternative Delivery and Time Windows (CVRPPADTW): A Hybrid Approach. In: *Proceedings of the Distributed Computing and Artificial Intelligence, 16th International Conference*, Special Sessions, pp. 33–40 https://doi.org/10.1007/978-3-030-23946-6_4 (Avila, Spain, 2019).
3. Heubeck, L., Hartwich, F. & Bocklisch, F. To share or not to share-expected transportation mode changes given different types of fully automated vehicles. *Sustainability* **15**, 5056. https://doi.org/10.3390/su15065056 (2023).
4. Chodorek, A. & Chodorek, R.R. Work-in-Progress: A Browser-Driven Sensor Service for Embedded IoT. In: *Proceedings of the 2022 International Conference on Embedded Software (EMSOFT)*, Shanghai, China, pp. 15–16. https://doi.org/10.1109/EMSOFT55006.2022.00015 (2022).
5. Chodorek, A., Chodorek, R. R. & Sitek, P. UAV-based and WebRTC-based open universal framework to monitor urban and industrial areas. *Sensors* **21**, 4061. https://doi.org/10.3390/s21124061 (2021).
6. Dameri, R.P. & Cocchia, A. Smart city and digital city: Twenty years of terminology evolution. In *Proceedings of the ItAIS 2013, X Conference of the Italian Chapter of AIS*, Milano, Italy. http://www.itais.org/proceedings/itais2013/pdf/119.pdf (2013).
7. Hall, P. Creative cities and economic development. *Urban Stud.* **37**, 639–649. https://doi.org/10.1080/00420980050003946 (2000).
8. Halegoua, G.R. Smart Cities; The MIT Press: Cambridge, UK. ISBN 9780262356848. (2020).
9. Peyman, M. et al. Edge computing and IoT analytics for agile optimization in intelligent transportation systems. *Energies* **14**, 6309. https://doi.org/10.3390/en14196309 (2021).
10. Rico, D. & Merino, P. A survey of end-to-end solutions for reliable low-latency communications in 5G networks. *IEEE Access* **8**, 192808–192834. https://doi.org/10.1109/ACCESS.2020.3032726 (2020).
11. Purucker, P., Schmid, J., Höß, A. & Schuller, B.W. System Requirements Specification for Unmanned Aerial Vehicle (UAV) to Server Communication. In: *Proceedings of the 2021 International Conference on Unmanned Aircraft Systems (ICUAS)*, Athens, Greece, pp. 1499–1508. https://doi.org/10.1109/ICUAS51884.2021.9476799 (2021).
12. Mourtzis, D., Angelopoulos, J. & Panopoulos, N. Smart manufacturing and tactile internet based on 5G in industry 4.0: challenges, applications and new trends. *Electronics* **10**, 3175. https://doi.org/10.3390/electronics10243175 (2021).
13. Gourisetti, S. N. G., Bhadra, S., Sebastian-Cardenas, D. J., Touhiduzzaman, M. & Ahmed, O. A Theoretical open architecture framework and technology stack for digital twins in energy sector applications. *Energies* **16**, 4853. https://doi.org/10.3390/en16134853 (2023).
14. Szpilko, D. et al. Energy in smart cities: technological trends and prospects. *Energies* **17**, 6439. https://doi.org/10.3390/en17246439 (2024).
15. Liang, S., Jin, S. & Chen, Y. A Review of Edge Computing Technology and Its Applications in Power Systems. *Energies* **17**, 3230. https://doi.org/10.3390/en17133230 (2024).
16. Sulieman, N. A., Ricciardi Celsi, L., Li, W., Zomaya, A. & Villari, M. Edge-oriented computing: a survey on research and use cases. *Energies* **15**, 452. https://doi.org/10.3390/en15020452 (2022).
17. Suer, M.-T., Jose, P. & Tchouankem, H. Experimental Evaluation of IEEE 802.11ax - Low Latency and High Reliability with Wi-Fi 6?. In: *Proceedings of the GLOBECOM 2022–2022 IEEE Global Communications Conference*, Rio de Janeiro, Brazil, pp. 377–382. https://doi.org/10.1109/GLOBECOM48099.2022.10001475. (2022).
18. Chodorek, A. & Chodorek, R. R. Web real-time communications-based unmanned-aerial-vehicle-borne internet of things and stringent time sensitivity: A case study. *Sensors* **25**, 524. https://doi.org/10.3390/s25020524 (2025).
19. Chourabi, H., Nam, T., Walker, S., Gil-Garcia, J.R., Mellouli, S., Nahon, K., Pardo, T.A. & Scholl, H.J. Understanding smart cities: An integrative framework. In *Proceedings of the 45th Hawaii International Conference on System Sciences*, Maui, HI, USA, pp. 2289–2297. https://doi.org/10.1109/HICSS.2012.615 (2012).
20. Gibson, D.V., Kozmetsky, G. & Smilor, R.W. (Eds.) The Technopolis Phenomenon: Smart Cities, Fast Systems, Global Networks; Rowman & Littlefield Publishers Inc: Lanham, MD, USA, ISBN: 0847677583 (1992).
21. Nastjuk, I., Trang, S. & Papageorgiou, E. I. Smart cities and smart governance models for future cities: current research and future directions. *Electron. Mark.* **32**, 1917–1924. https://doi.org/10.1007/s12525-022-00609-0 (2022).
22. Yigitcanlar, T. et al. Understanding 'smart cities': Intertwining development drivers with desired outcomes in a multidimensional framework. *Cities* **81**, 145–160. https://doi.org/10.1016/j.cities.2018.04.003 (2018).
23. Piro, G., Cianci, I., Grieco, L. A., Boggia, G. & Camarda, P. Information centric services in smart cities. *J. Syst. Softw.* **88**, 169–188. https://doi.org/10.1016/j.jss.2013.10.029 (2014).
24. Smart cities, European Commission. https://commission.europa.eu/eu-regional-and-urban-development/topics/cities-and-urban-development/city-initiatives/smart-cities_en (2024).
25. Mongo, M. & Daidj, N. Digital twin: a driver of sustainable smart cities? Evidence from a bibliometric analysis. *Environ. Econ. Policy Stud.* 1–29. https://doi.org/10.1007/s10018-025-00437-4 (2025).
26. Long, Y., Bao, Y. & Zeng, L. Research on edge-computing-based high concurrency and availability "cloud, edge, and end collaboration" substation operation support system and applications. *Energies* **17**, 194. https://doi.org/10.3390/en17010194 (2024).
27. Jaroń, A., Borucka, A., Deliś, P. & Sekrecka, A. An assessment of the possibility of using unmanned aerial vehicles to identify and map air pollution from infrastructure emissions. *Energies* **17**, 577. https://doi.org/10.3390/en17030577 (2024).
28. Chodorek, A., Chodorek, R. R. & Yastrebov, A. The prototype monitoring system for pollution sensing and online visualization with the use of a UAV and a WebRTC-based platform. *Sensors* **22**, 1578. https://doi.org/10.3390/s22041578 (2022).
29. Chodorek, A., Chodorek, R. R. & Sitek, P. Response time and intrinsic information quality as criteria for the selection of low-cost sensors for use in mobile weather stations. *Electronics* **11**, 2448. https://doi.org/10.3390/electronics11152448 (2022).
30. Almalki, F. A., Soufiene, B. O., Alsamhi, S. H. & Sakli, H. A low-cost platform for environmental smart farming monitoring system based on IoT and UAVs. *Sustainability* **13**, 5908. https://doi.org/10.3390/su13115908 (2021).

31. Elloumi, M., Dhaou, R., Escrig, B., Idoudi, H. & Saidane, L.A. Monitoring road traffic with a UAV-based system. In *Proceedings of the 2018 IEEE Wireless Communications and Networking Conference (WCNC)*, Barcelona, Spain, pp. 1–6.https://doi.org/10.1109/WCNC.2018.8377077 (2018).
32. Bisio, I., Garibotto, C., Haleem, H., Lavagetto, F. & Sciarrone, A. A Systematic review of drone based road traffic monitoring system. *IEEE Access* **10**, 101537–101555. https://doi.org/10.1109/ACCESS.2022.3207282 (2022).
33. Mohammed, A., Erbad, A., Nahom, H., Albaseer, A., Abdallah, M. & Guizani, M. FDRL Approach for Association and Resource Allocation in Multi-UAV Air-To-Ground IoMT Network. In *Proceedings of the GLOBECOM 2022–2022 IEEE Global Communications Conference*, Rio de Janeiro, Brazil, pp. 1417–1422. https://doi.org/10.1109/GLOBECOM48099.2022.10000670 (2022).
34. Han, M., Lee, J., Rim, M. & Kang, C. G. Dynamic bandwidth part allocation in 5G ultra reliable low latency communication for unmanned aerial vehicles with high data rate traffic. *Sensors* **21**, 1308. https://doi.org/10.3390/s21041308 (2021).
35. Giordani, M., Polese, M., Mezzavilla, M., Rangan, S. & Zorzi, M. Toward 6G Networks: Use cases and technologies. *IEEE Commun. Mag.* **58**, 55–61. https://doi.org/10.1109/MCOM.001.1900411 (2020).
36. Gour, R. & Yadav, S. The Transformative Role of UAVs in Smart City Frameworks. In: *Internet of Vehicles and Computer Vision Solutions for Smart City Transformations*; (eds. Abraham, A., Prasad, S., Alhammadi, A., Lestable, T., Chaabane, F.) Lecture Notes in Intelligent Transportation and Infrastructure, pp. 3–24, https://doi.org/10.1007/978-3-031-72959-1_1 (Springer, Cham, Switzerland, 2025).
37. Schulz, P. et al. Latency Critical IoT Applications in 5G: Perspective on the Design of Radio Interface and Network Architecture. *IEEE Commun. Mag.* **55**, 70–78. https://doi.org/10.1109/MCOM.2017.1600435CM (2017).
38. Adame, T., Carrascosa-Zamacois, M. & Bellalta, B. Time-Sensitive Networking in IEEE 802.11be: On the Way to Low-Latency WiFi 7. *Sensors* **21**, 4954. https://doi.org/10.3390/s21154954 (2021).
39. Wireless technologies and Use Cases in Industrial IOT. https://www.ciscolive.com/c/dam/r/ciscolive/emea/docs/2020/pdf/BRKIOT-1775.pdf (2020).
40. Kang, Y., Lee, S., Gwak, S., Kim, T. & An, D. Time-sensitive networking technologies for industrial automation in wireless communication systems. *Energies* **14**, 4497. https://doi.org/10.3390/en14154497 (2021).
41. Medeiros, D.d.F., Souza, C.P.d., Carvalho, F.B.S.d & Lopes, W.T.A. Energy-Saving Routing Protocols for Smart Cities. *Energies* **15**, 7382 https://doi.org/10.3390/en15197382 (2022).
42. EL-Garoui, L., Pierre, S. & Chamberland, S. A new SDN-based routing protocol for improving delay in smart city environments. *Smart Cities* **3**, 1004–1021. https://doi.org/10.3390/smartcities3030050 (2020).
43. Botez, R., Pasca, A.-G., Sferle, A.-T., Ivanciu, I.-A. & Dobrota, V. Efficient network slicing with SDN and Heuristic algorithm for low latency services in 5G/B5G networks. *Sensors* **23**, 6053. https://doi.org/10.3390/s23136053 (2023).
44. Martalò, M., Pettorru, G. & Atzori, L. A Cross-layer survey on secure and low-latency communications in next-generation IoT. *IEEE Trans. Netw. Serv. Manag.* **21**, 4669–4685. https://doi.org/10.1109/TNSM.2024.3390543 (2024).
45. Lee, S. & An, D. Enhanced flow control for low latency in QUIC. *Energies* **15**, 4241. https://doi.org/10.3390/en15124241 (2022).
46. Jeddou, S. et al. Delay and energy consumption of MQTT over QUIC: An empirical characterization using commercial-off-the-shelf devices. *Sensors* **22**, 3694. https://doi.org/10.3390/s22103694 (2022).
47. Black, D. G., Andjelic, D. & Salcudean, S. E. Evaluation of communication and human response latency for (human) teleoperation. *IEEE Trans. Med. Robot. Bionics* **6**, 53–63. https://doi.org/10.1109/TMRB.2024.3349612 (2024).
48. Stewart, R., Tüxen, M. & Nielsen, K. Stream Control Transmission Protocol. RFC 9260. https://www.rfc-editor.org/info/rfc9260 (2022).
49. Mahmoud, H. & Abozariba, R. A systematic review on WebRTC for potential applications and challenges beyond audio video streaming. *Multimed Tools Appl.* **84**, 2909–2946. https://doi.org/10.1007/s11042-024-20448-9 (2025).
50. Web of Things (WoT) Architecture 1.1 W3C Recommendation. https://www.w3.org/TR/wot-architecture11/ (2023).
51. WebRTC Extended Use Cases W3C Group Draft Note. https://www.w3.org/TR/webrtc-nv-use-cases/ (2023).
52. Patel, C. & Doshi, N. A novel MQTT security framework in generic IoT model. *Procedia Comput. Sci.* **171**, 1399–1408. https://doi.org/10.1016/j.procs.2020.04.150 (2020).
53. Fernández, F. et al. Even lower latency in IIoT: Evaluation of QUIC in industrial IoT scenarios. *Sensors* **21**, 5737. https://doi.org/10.3390/s21175737 (2021).
54. Boeding, M., Scalise, P., Hempel, M., Sharif, H. & Lopez, J., Jr. Toward wireless smart grid communications: an evaluation of protocol latencies in an open-source 5G testbed. *Energies* **17**, 373 https://doi.org/10.3390/en17020373 (2024).
55. Boeding, M., Scalise, P., Hempel, M. & Sharif, H. Evaluating the Latency Impact for Time-Critical Operational Technology Applications of Transitioning IEC-61850 GOOSE Operations to 5G. In *Proceedings of the 2024 IEEE 20th Consumer Communications & Networking Conference (CCNC)*, Las Vegas, NV, USA, pp. 1–2. https://doi.org/10.1109/CCNC51664.2024.10454815 (2024).
56. Jiang, S., Zhang, Q., Wu, A., Liu, Q., Wu, J.; & Xia, P. A low-latency reliable transport solution for network-connected UAV. In *Proceedings of the 2018 10th International Conference on Communication Software and Networks*, Ponta Delgada, Portugal, pp. 511–515. https://doi.org/10.1109/ICCSN.2018.8488271 (2018).
57. Kilic, F., Hassan, M. & Hardt, W. Prototype for multi-UAV monitoring-control system using WebRTC. *Drones* **8**, 551. https://doi.org/10.3390/drones8100551 (2024).
58. Holmberg, C., Hakansson, S. & Eriksson, G. Web Real-Time Communication Use Cases and Requirements. RFC 7478. https://www.rfc-editor.org/rfc/rfc7478.html (2015).
59. Tiberkak, A., Hentout, A. & Belkhir, A. WebRTC-based MOSR remote control of mobile manipulators. *Int. J. Intell. Robot. Appl.* **7**, 304–320. https://doi.org/10.1007/s41315-023-00281-3 (2023).
60. OpenStreetMap. https://www.openstreetmap.org/ (2025).

Author contributions

Conceptualization, A.Ch. and R.Ch.; formal analysis, A.Ch.; investigation, R.Ch.; software, A.Ch. and R.Ch.; visualization, A.Ch.; writing – original draft, A.Ch. and R.Ch.; writing – review & editing, P.S.; funding acquisition, P.S. All authors reviewed the manuscript.

Funding

This research was funded by the Polish Ministry of Science and Higher Education.

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to R.R.C.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2026